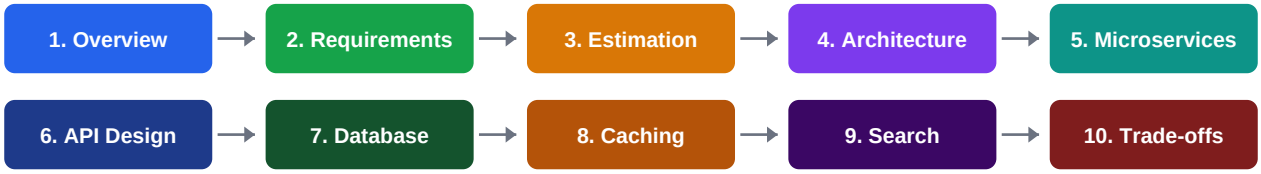
E-Commerce Platform

Design a system like **Amazon** / **Flipkart** / **Shopify** from scratch



This PDF is your complete session guide. Each section maps to one slide. Every page includes tables, diagrams, talking points, and reference links to deepen your understanding.

Reference Links (clickable)

Amazon Architecture — All Things Distributed (https://www.allthingsdistributed.com)

System Design Primer (GitHub) (https://github.com/donnemartin/system-design-primer)

High Scalability Blog (http://highscalability.com)

AWS Architecture Center (https://aws.amazon.com/architecture)

Netflix Tech Blog (https://netflixtechblog.com)

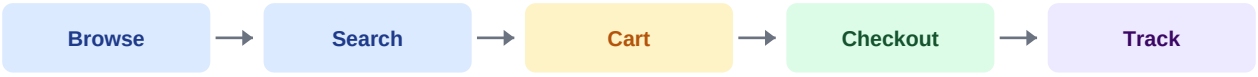
Section 1 of 10

Overview — What Are We Building?

Section 1

An e-commerce platform lets users browse products, add to cart, place orders, and pay. Behind that simple flow sits one of the most complex distributed systems ever built.

Core user journeys



Dimension	Why challenging
Scale	Amazon handles 66,000 orders/hour on normal days; millions during sales
Consistency	Inventory must be accurate — overselling = angry customers
Latency	100 ms delay costs 1% in sales (Amazon research)
Reliability	1 hour of downtime during Prime Day = $100M+ lost
Variety	Products, users, payments, logistics — each needs its own system

Talking point: Start by explaining the business impact. System design is not academic — every millisecond and every downtime minute has a dollar value.

Learn more:

How Amazon.com Works (HowStuffWorks) (https://computer.howstuffworks.com/internet/basics/amazon.htm)

Amazon Architecture (High Scalability) (http://highscalability.com/amazon-architecture)

Section 2 of 10

Requirements

Section 2

Always clarify scope first. An e-commerce system has dozens of features — agree on the MVP before drawing any boxes.

Functional Requirements

- User registration and login (OAuth supported)
- Product catalogue: browse, search, filter
- Product detail pages with images and reviews
- Shopping cart (persistent across sessions)
- Checkout: address, payment, order confirmation
- Order tracking and order history
- Seller portal: list and manage products
- Inventory management: stock levels
- Notifications: email/SMS for order events

Non-Functional Requirements

- Availability: 99.99% (< 1hr downtime/year)
- Latency: product page < 200ms, search < 500ms
- Consistency: inventory must be strongly consistent
- Scalability: handle 10x traffic spikes (flash sales)
- Durability: no orders or payment data ever lost
- Security: PCI-DSS compliance for payments
- Fault tolerance: one service failure must not cascade

Out of scope (for this session)

Recommendation engine, fraud detection ML, marketplace seller financials, returns/refunds flow, warehouse management. Mention these to show breadth, then park them.

Talking point: Interviewers often use requirements to trap candidates into designing too much. Scope-limiting early is a sign of engineering maturity.

Section 3 of 10

Capacity Estimation

Section 3

Assume 10M daily active users, 100M product catalogue, 1M orders/day.

~700/sec
READ QPS (PAGES/SEARCH)

~12/sec
WRITE QPS (ORDERS PLACED)

~50 TB
PRODUCT IMAGE STORAGE

~2 TB/day
EVENT / LOG DATA

Metric	Assumption	Calculation	Result
Page reads/sec	10M DAU × 6 pages/day	60M / 86400	~700 QPS
Orders/sec	1M orders/day	1M / 86400	~12/sec
Peak (10x)	Flash sale spike	12 × 10	~120/sec
Product images	100M products × 5 images × 100KB	100M×500KB	~50 TB
DB storage	100M products × 5KB metadata	100M × 5KB	~500 GB
Order records	1M/day × 365 × 5 years	1.8B records	~1.8 TB

Key insight: Product reads vastly outnumber writes (read:write ~ 60:1). Images dominate storage (CDN required). Orders need strong consistency. These three facts shape the entire architecture.

Talking point: Flash sale spikes (10x) are the most dangerous. Design for peak, not average. Auto-scaling groups and queue-based order processing smooth these spikes.

Section 4 of 10

High-Level Architecture

Section 4

An e-commerce platform is a collection of microservices, each owning a business domain, communicating via APIs and message queues.

Layer	Component	Role
Edge	CDN (CloudFront/Akamai)	Cache static assets (images, JS, CSS) at 200+ global edge nodes
Gateway	API Gateway	Auth, rate limiting, routing, SSL termination, request logging
Routing	Load Balancer (ALB)	Distribute traffic to healthy service instances
Services	Microservices (10+)	Each owns a domain: users, products, orders, payments, inventory…
Async	Message Queue (Kafka/SQS)	Decouple services — order placed event fans out to 5 consumers
Cache	Redis Cluster	Session data, cart, hot product pages, rate limit counters
Storage	PostgreSQL + DynamoDB + S3	Relational for orders, NoSQL for catalogue, S3 for images
Search	Elasticsearch	Full-text product search with filters, facets, ranking
Observability	Prometheus + Grafana	Metrics, alerts, dashboards for every service

Talking point: The API Gateway is the single entry point. It handles auth so individual services do not have to. This single responsibility makes services simpler and more secure.

AWS Architecture — E-Commerce Reference (https://aws.amazon.com/architecture/reference-architecture-diagrams/)

Section 5 of 10

Microservices Breakdown

Section 5

Each service owns its data, its API, and its deployment. No shared databases between services.

Service	Responsibilities	DB Choice	Key APIs
User Service	Registration, login, profiles, addresses	PostgreSQL	POST /register, POST /login, GET /profile
Product Service	Catalogue, categories, images, attributes	DynamoDB + S3	GET /products, GET /product/:id
Inventory Service	Stock levels, reservations, warehouse	PostgreSQL	GET /stock/:id, POST /reserve
Cart Service	Add/remove items, persist across sessions	Redis	POST /cart/add, GET /cart, DELETE /cart/item
Order Service	Create, track, cancel orders; state machine	PostgreSQL	POST /order, GET /order/:id, GET /orders
Payment Service	Charge, refund, payment methods, PCI scope	PostgreSQL	POST /charge, POST /refund
Notification Svc	Email, SMS, push for order/shipping events	Kafka consumer	Event-driven, no public API
Search Service	Full-text search, filters, autocomplete	Elasticsearch	GET /search?q=&filters;=
Review Service	Product ratings, reviews, moderation	PostgreSQL	POST /review, GET /reviews/:product_id
Recommendation	Personalised product suggestions	ML model + Redis	GET /recommendations/:user_id

Inter-service communication

Synchronous (REST/gRPC): User → Cart → Inventory → Order → Payment (checkout flow — needs immediate response).

Asynchronous (Kafka events): Order Service publishes OrderPlaced event → Notification, Inventory, Analytics all consume independently.

Talking point: The Cart Service uses Redis (in-memory) because carts are temporary and must be blazing fast. The Order Service uses PostgreSQL because orders are permanent and need ACID transactions.

Microservices patterns — martinfowler.com (https://martinfowler.com/articles/microservices.html)

Section 6 of 10

API Design

Section 6

RESTful APIs with versioning. Critical flows: product browse, add to cart, checkout.

Browse & Search

```
GET /api/v1/products?category=electronics&sort;=price&page;=1
GET /api/v1/search?q=iphone+15&filters;=brand:apple,price:50000-80000
GET /api/v1/products/:id
```

Checkout Flow

```
POST /api/v1/orders
{ cart_id, address_id, payment_method_id }
Response 201:
{ order_id, status: "pending", total, payment_url }
```

Cart Operations

```
POST /api/v1/cart/items { product_id, qty }
PUT /api/v1/cart/items/:id { qty: 3 }
DELETE /api/v1/cart/items/:id
GET /api/v1/cart
```

Order Tracking

```
GET /api/v1/orders/:id
Response: { status: "shipped", tracking_id, eta }
GET /api/v1/orders (list, paginated)
```

API design principles

Principle	Example	Why
Versioning	/api/v1/...	Old clients continue working after API updates
Pagination	?page=2&limit;=20	Never return unbounded lists — DB and client both suffer
Idempotency	Idempotency-Key header	Retry safe — double-clicking checkout should not charge twice
HATEOAS links	"links": {"track": "/orders/1/track"}	API is self-documenting, clients navigate by links

Talking point: Idempotency is critical for payments. If a network drop happens mid-checkout, the app retries. Without idempotency keys, the user gets charged twice. Stripe and Razorpay both require this header.

Stripe API Idempotency Keys (real-world example) (https://stripe.com/docs/api/idempotent_requests)

Section 7 of 10

Database Design

Section 7

Different data needs different databases. One size does not fit all in e-commerce.

Data Domain	DB Type	Why this choice	Example
Users & Orders	PostgreSQL (SQL)	Needs ACID — money transactions must be consistent	Amazon RDS
Product catalogue	DynamoDB (NoSQL)	100M+ products, flexible attributes, fast key-value reads	AWS DynamoDB
Product images	Object Store (S3)	Cheap, durable, CDN-integrated blob storage	AWS S3
Sessions & Cart	Redis (in-memory)	Millisecond access, TTL-based expiry, simple data	ElastiCache
Search index	Elasticsearch	Full-text search, facets, ranking — SQL cannot do this well	AWS OpenSearch
Analytics events	ClickHouse / Redshift	Columnar storage for fast aggregation queries	AWS Redshift
Audit / payments	PostgreSQL + append-only	Payment records are never updated, only appended	Write-once tables

Critical: Inventory consistency

Inventory is the hardest consistency problem. Two users must not buy the last item simultaneously. Solution: use database-level row locking (SELECT FOR UPDATE) or optimistic concurrency with version numbers.

```
UPDATE inventory SET stock = stock - 1, version = version + 1 WHERE product_id = ? AND stock > 0 AND version = ?
```

If this UPDATE affects 0 rows → version mismatch → retry or show "out of stock". This prevents overselling without distributed locks.

Talking point: Mention that payment tables are append-only (like accounting ledgers). You never UPDATE a payment row — you INSERT a new record. This makes auditing trivial and prevents data loss.

PostgreSQL row locking patterns (https://www.postgresql.org/docs/current/explicit-locking.html)

DynamoDB best practices — AWS (https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html)

Section 8 of 10

Caching Strategy

Section 8

E-commerce has multiple caching layers. Product pages, sessions, inventory counts, and API responses all benefit from caching.

Cache Layer	What is cached	TTL	Eviction
CDN (CloudFront)	Static assets: images, CSS, JS, fonts	24–72 hrs	Cache-Control headers
CDN (HTML pages)	Product listing pages for non-logged users	5 min	Vary: Cookie header
Redis — Sessions	User session tokens and auth state	30 min	TTL auto-expire
Redis — Cart	Shopping cart contents per user	7 days	TTL + LRU
Redis — Product	Hot product detail pages (JSON)	10 min	LRU
Redis — Inventory	Stock counts for fast availability checks	30 sec	Write-through
API Gateway cache	GET /products responses for anonymous users	60 sec	Query-key based

Cache invalidation strategy

- Write-through: update cache and DB simultaneously (inventory stock counts — must stay accurate)
- Cache-aside: read from cache, on miss query DB and populate cache (product pages)
- Event-driven invalidation: when a product is updated, publish event → cache service clears relevant keys

Talking point: Cache invalidation is famously the hardest problem in computer science. In e-commerce, stale inventory counts are acceptable for 30 seconds. Stale payment data is never acceptable. TTL strategy must match business tolerance.

Redis caching patterns (https://redis.io/docs/manual/patterns/)

Section 9 of 10

Search & Discovery

Section 9

Search is the highest-value feature in e-commerce. 34% of users go directly to the search bar (Baymard Institute).

Feature	How it works	Tech
Full-text search	Inverted index on product name, description, brand, tags	Elasticsearch
Autocomplete	Prefix queries on product names — results in <50ms	Elasticsearch completion suggester
Faceted filters	Brand, price range, rating, colour — aggregation queries	Elasticsearch aggs
Ranking / relevance	Boost by sales velocity, rating, stock availability	Elasticsearch function score
Typo tolerance	Fuzzy matching — "iphne" → "iphone"	Elasticsearch fuzziness
Personalisation	User history re-ranks results using click-through data	ML model + Redis
Sync to ES	Product updates published to Kafka → Logstash → ES index	Kafka + Logstash

Talking point: Never put search on the product database. A SELECT LIKE %iphone% on a 100M-row table will kill your DB. Elasticsearch maintains a separate inverted index, built for this exact workload.

Elasticsearch e-commerce guide (https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html)

Algolia e-commerce search patterns (https://www.algolia.com/industries-and-solutions/ecommerce/)

Section 10 of 10

Trade-offs & Design Decisions

Section 10

E-commerce is full of tension between consistency vs availability, monolith vs microservices, build vs buy.

Decision	Choice	Why	Trade-off
Architecture	Microservices	Independent scale and deploy per service	Operational complexity, distributed tracing needed
Payment DB	SQL + ACID	Money must be consistent — no eventual consistency here	Harder to scale horizontally
Inventory	Optimistic locking	No distributed locks, high throughput	Retry logic needed on contention
Cart storage	Redis	Blazing fast, TTL-based cleanup	Lost on Redis failure without persistence
Search	Elasticsearch	Purpose-built, rich features	Eventual sync lag from product DB
Images	S3 + CDN	Cheap, durable, globally fast	CDN cache invalidation is slow on updates
Async events	Kafka	Durable, replay-able, fan-out to N consumers	More infra to manage vs simple HTTP calls
Payments	Third-party (Stripe/Razorpay)	PCI-DSS handled by vendor, fast integration	Vendor lock-in, fees per transaction

The hardest problems in e-commerce

1. Flash sale inventory: 50,000 users racing to buy 100 items simultaneously. Solution: queue-based purchase flow + Redis atomic DECR.
2. Payment idempotency: network drops during checkout. Solution: idempotency keys on payment API + distributed transaction (Saga pattern).
3. Catalog search freshness: product updated in DB but old data shows in search for 30 sec. Solution: Kafka event → instant ES update.

Talking point: Close your session with "what breaks first at 10x scale?" Answer: the monolithic parts — shared databases, synchronous calls in the checkout path. The fix is always the same: decompose, cache, and go async.

Reference links for deeper reading

System Design Primer — E-Commerce patterns (GitHub) (https://github.com/donnemartin/system-design-primer)

Shopify Engineering Blog (https://shopify.engineering)

Amazon DynamoDB — How we built it (https://www.allthingsdistributed.com/2007/10/amazons_dynamo.html)

Saga pattern for distributed transactions (https://microservices.io/patterns/data/saga.html)

Kafka use cases — Confluent (https://www.confluent.io/learn/apache-kafka/)

Redis e-commerce patterns (https://redis.com/solutions/use-cases/ecommerce/)

Stripe payment processing best practices (https://stripe.com/docs/payments)

Elasticsearch product search tutorial (https://www.elastic.co/guide/en/elasticsearch/reference/current/search-your-data.html)

AWS Well-Architected Framework for e-commerce (https://aws.amazon.com/architecture/well-architected/)

[High Scalability — how Shopify handles millions of req/sec](http://highscalability.com/blog/2021/11/8/shopify-lessons-learned-from-going-through-shopocalypse.html)